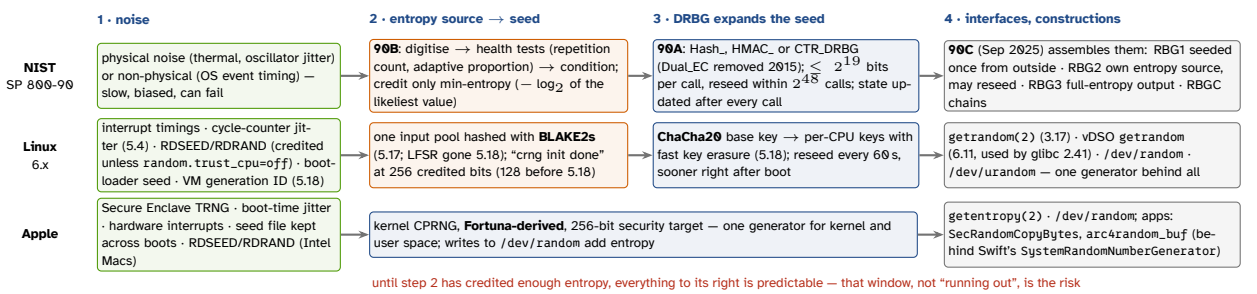


In one line: A few hundred bits of real **entropy** seed a **DRBG** that expands them into an unlimited, unpredictable stream. Take every key, nonce, IV and token from the **OS generator** (getrandom, SecRandomCopyBytes, SecureRandom, crypto.getRandomValues); the dangerous moment is *before* it is seeded, or after its state was copied.



PRNG · CSPRNG · TRNG

- PRNG:** a fast deterministic function of its state, built for statistics. Mersenne Twister (Python random, mt19937): state = 624 words, output tempering is invertible, so 624 outputs give every later one. V8's Math.random (Chrome 49+) is xorshift128+ — "still not cryptographically secure"; Go math/rand/v2 "might be easily predictable".
- CSPRNG / DRBG:** next bit unguessable from all earlier output; **backtracking resistance** — a stolen state does not reveal past output; **prediction resistance** — a stolen state stops predicting the future — only by reseeding with fresh entropy.
- TRNG / entropy source:** physical noise; slow, biased, may fail silently — tested, conditioned, used only to seed.
- Once seeded with 256 bits a generator does not "run out". Linux 5.6 removed the blocking pool: the CRNG output is "good enough to use even for key generation".

Linux — what blocks, today

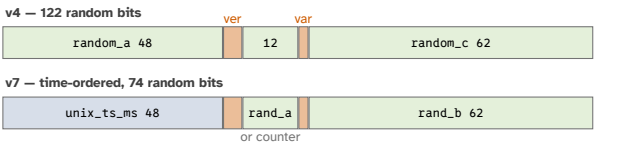
call	before CRNG ready	after
getrandom(b, n, 0)	blocks	CSPRNG bytes
+ GRND_NONBLOCK	→ 1, EAGAIN	same
+ GRND_INSECURE (5.6)	returns anyway — not for secrets	same
+ GRND_RANDOM	no-op since 5.6 (was the blocking pool)	same
/dev/random	blocks (pre-5.6 also whenever the estimate ran low)	same
/dev/urandom	never blocks (= GRND_INSECURE; tries jitter first)	same

Reads ≤ 256 B are never cut short by a signal. random(7) (man-pages 6.19, 2026) still describes the pre-5.6 blocking pool.

When seeding goes wrong

- First boot, headless:** no disk, keyboard or mouse noise; keys generated before the pool is seeded (routers, 2012 below).
- Cloned VM / restored snapshot:** two machines, one generator state. VM generation ID (5.18) forces a reseed; WireGuard wipes its keys on VM fork.
- fork():** the child inherits a user-space generator's state — Rust ThreadRng (ChaCha12, reseeds every 64 kB) does not reseed on fork; call reseed().
- Microcontrollers:** ESP-IDF esp_random() is true random only while Wi-Fi or Bluetooth runs (or bootloader_random_enable()), else pseudo-random — seed an mbedTLS DRBG from it.

UUIDs — identifiers, not secrets (RFC 9562)



May 2024, obsoletes RFC 4122; "SHOULD utilize UUIDv7 instead of UUIDv1 and UUIDv6". v4 has "poor database-index locality"; v7 sorts by creation (ms) and reveals it; a counter or sub-ms time in rand_a costs random bits. "SHOULD NOT assume that UUIDs are hard to guess", "MUST NOT be used as security capabilities" — no reset tokens, no share links. CSPRNG "SHOULD" generate them.

Modulo bias

a random byte: 25 full runs of 0-9 ...

byte % 10: digits 0-5 have 26/256, 6-9 have 25/256 — 4 % likelier. Reject the red tail and redraw, or call Int.random(in:), arc4random_uniform, randomInt, secrets.randbelow.

```
const lim = 2**32 - 2**32 % n; // largest multiple of n; u = new Uint32Array(1)
do crypto.getRandomValues(u); while (u[0] >= lim); return u[0] % n;
```

Related: block-cipher-modes (IV vs nonce) · public-key-maths (ECDSA k) · encoding-encryption-hashing-signing · Hashing — tables, crypto, rings, passwords, Bloom (birthday bound) · Cryptography with CryptoKit · Keychain · Secure Enclave · biometrics · JWT attacks & token hardening

Which call, per platform

	use	know
Apple	SecRandomCopyBytes; Int.random(in:); CryptoKit SymmetricKey(size:) new SecureRandom() — no seed, no provider (JDK on Linux: NativePRNG, /dev/urandom)	check errSecSuccess; random(in:) is unbiased (Lemire); GameplayKit sources "not cryptographically robust" JDK getInstanceStrong() = NativePRNGBlocking (/dev/random); "Crypto" SHA1PRNG provider gone in Android 9; java.util.Random not crypto $\leq 65\,536$ B a call, else QuotaExceededError; integer arrays only; randomUUID needs a secure context
Java, Android	crypto.getRandomValues, crypto.randomUUID	randomInt: max — min $< 2^{48}$; randomUUID caches entropy for 128 UUIDs
Browser	react-native-get-random-values (imported first, before uuid) or expo-crypto	bridges to SecRandomCopyBytes / SecureRandom; Expo getRandomBytes ≤ 1024 B and falls back to Math.random in development
Node	secrets.token_urlsafe() etc. (3.6)	default 32 B — "sufficient" as of 2015; random is for "modelling and simulation"
React Native	crypto/rand.Read, rand.Text()	1.24: Read never returns an error (crashes instead); Text = base32, ≥ 128 bits
Python	ProcessPrng, BCryptGenRandom	what Go and Swift call

Keys, nonces, IVs, tokens

symmetric key	128 / 256 bits straight from the CSPRNG, or a KDF of a secret — never a password or the time
CBC IV	unpredictable: fresh random 16 B per message (SP 800-38A)
GCM nonce	unique per key: a counter, or random 96-bit for $\leq 2^{32}$ messages per key (SP 800-38D); XChaCha20 takes a random 192-bit nonce
ECDSA k	a repeated or guessable k gives away the key — RFC 6979 derives it deterministically; Ed25519 (RFC 8032) is deterministic by design
access token, reset link	OAuth: guessing $\leq 2 - 128$ (MUST), $\leq 2 - 160$ (SHOULD) — i.e. ≥ 128 bits of CSPRNG output, base64url
session ID	OWASP: ≥ 64 bits of entropy (≥ 16 hex chars) — 585 years at $10\,000$ guesses/s vs $100\,000$ live sessions

Failures — one weak generator, every key

2006-08 Debian	a Debian-only OpenSSL patch (0.9.8c-1, Sep 2006) left the PID as the only input: 32 767 keys per architecture, size and type. SSH, OpenVPN, DNSSEC, X.509 keys; every DSA key used there (CVE-2008-0166)
2012 Ps & Qs	routers, firewalls seeded at boot → shared RSA primes, batch GCD: keys of 0.50 % of TLS and 0.03 % of SSH hosts; DSA keys of 1.03 % of SSH hosts by repeated k
2013 Android	Harmony SHA1PRNG bug + unseeded OpenSSL PRNG (≤ 4.3 , CVE-2013-7372) → Bitcoin wallets repeated ECDSA k, coins stolen; fixed in 4.4
2013-15 DualEC	whoever knows e with $P = eQ$ predicts all later output; NIST advised against it in Sep 2013. Juniper ScreenOS (Dec 2015): intruders swapped Q → passive VPN decryption
2023 Milk Sad	bx seed (Libbitcoin Explorer 3.x): mt19937 seeded with 32 bits of time → 2^{32} wallets, drained in the wild (CVE-2023-39910)
2025 Zen 5	16/32-bit RDSEED returns 0 while signalling success (AMD-SB-7055); Linux disables it on unfixed microcode

Traps

- Ignoring SecRandomCopyBytes' status: on failure whatever the buffer held — usually zeros — becomes the key.
- Seeding any generator with the time, the PID or a test constant — 32 bits is 2^{32} guesses.
- Hashing the output of a weak PRNG: SHA-256 of 32 bits of entropy is still 32 bits.
- A 6-digit OTP holds $\log_2 10^6 \approx 20$ bits — safe only with rate limits and expiry, never as a bearer token.

Remember

Entropy seeds, the DRBG expands, the OS serves. Keys random, nonces unique, IVs unpredictable, tokens ≥ 128 bits. Watch first boot, clones and forks — UUIDs identify, never authorise.